

| Introduction to Python

Functions

A **function** is a reusable block of code that performs a specific task.

1. Function without parameters

A function does not have to receive any input.

```
def say_hello():  
    print("Hello!")
```

Call the function:

```
say_hello()
```

2. Function with parameters

A function can receive one or more **parameters**.

```
def greet(name):  
    print(f"Hello, {name}!")
```

When calling the function, the parameter is **required**:

```
greet("Ahmad")
```

If you call:

```
greet()
```

Python will raise an error because the required `name` argument was not provided.

3. Function with default parameters

A parameter can have a **default value**.

```
def greet(name="Guest"):  
    print(f"Hello, {name}!")
```

Now, you can call the function in two ways:

```
greet("Ahmad")
```

Output:

```
Hello, Ahmad!
```

Or without passing the argument:

```
greet()
```

Output:

```
Hello, Guest!
```

The default value is used when no value is provided.

Key idea

- **No parameters:** `def func():`
- **Required parameter:** `def func(name):` → must provide `name`
- **Default parameter:** `def func(name="Guest"):` → `name` can be omitted

```
In [1]: def this_is_a_function_with_no_args():  
        print("Hello!")  
        this_is_a_function_with_no_args()
```

```
Hello!
```

```
In [2]: def sum_values(a,b):  
        x = a+b  
        return x
```

```
In [3]: sum_values(2,3)
```

```
Out[3]: 5
```

```
In [ ]: def hello(name="George"):  
        print("Hello "+name)
```

```
hello("Maria")  
hello()
```

```
Hello Maria  
Hello George
```

```
In [ ]: def hello(name):  
        print("Hello "+name)
```

```
hello("Maria")  
hello()
```

```
Hello Maria
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In [7], line 5  
      2     print("Hello "+name)  
      4     hello("Maria")  
----> 5     hello()  
  
TypeError: hello() missing 1 required positional argument: 'name'
```

Tuples

A tuple is a data type that stores elements that are:

1. ordered
2. immutable (cannot be modified after creation)
3. allows duplicates

In AI and data processing, we often need to store data that:

- should not change
- represents fixed values
- groups related information together

For this purpose, Python provides the tuple.

```
In [12... models1 = ("CNN", "RNN", "Transformer")
print(models1)
('CNN', 'RNN', 'Transformer')
```

```
In [13... models2 = "CNN", "RNN", "Transformer"
models1 == models2
```

```
Out[13... True
```

```
In [15... len(models1)
```

```
Out[15... 3
```

Accessing Tuple Elements

```
In [17... models1[1]
```

```
Out[17... 'RNN'
```

Since tuples are indexed, they can have items with the same value:

```
In [18... fruitstuple = ("apple", "banana", "cherry", "apple", "cherry")
print(fruitstuple)
('apple', 'banana', 'cherry', 'apple', 'cherry')
```

Tuple items can be of any data type, and it can contain different data types:

```
In [19... tuple1 = ("abc", 34, True, 40, "male")
print(tuple1)
('abc', 34, True, 40, 'male')
```

Tuples cannot be modified after creation.

```
In [ ]: models = ("CNN", "RNN", "Transformer")
models[0] = "LSTM"
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In [24], line 3
      1 models = ("CNN", "RNN", "Transformer")
----> 3 models[0] = "LSTM"

TypeError: 'tuple' object does not support item assignment
```

Tuple Unpacking

```
In [25... width, height = (1920, 1080)

print(width)
print(height)
```

```
1920
1080
```

Add Items

1. Convert into a list
2. Add tuple to a tuple

```
In [8]: x = ("apple", "banana", "cherry")
y = list(x)
y.append("orange")
x = tuple(y)
```

```
In [9]: x
```

```
Out[9]: ('apple', 'banana', 'cherry', 'orange')
```

```
In [10... x = ("apple", "banana", "cherry")
y = ("orange",)
x += y

print(x)
```

```
('apple', 'banana', 'cherry', 'orange')
```

```
In [12... x = ("apple", "banana", "cherry")
y = ("orange",)
print(type(y))
x += y

print(x)
```

```
<class 'tuple'>
('apple', 'banana', 'cherry', 'orange')
```

Note: When creating a tuple with only one item, remember to include a comma after the item, otherwise it will not be identified as a tuple.

Remove Items

1. Convert the tuple into a list
2. Delete the tuple completely

```
In [13... thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.remove("apple")
thistuple = tuple(y)
```

```
In [14... thistuple
```

```
Out[14... ('banana', 'cherry')
```

```
In [29... thistuple = ("apple", "banana", "cherry")
del thistuple
print(thistuple) #this will raise an error because the tuple no longer exists
```

```
-----
NameError                                Traceback (most recent call last)
Cell In [29], line 3
      1 thistuple = ("apple", "banana", "cherry")
      2 del thistuple
----> 3 print(thistuple)

NameError: name 'thistuple' is not defined
```

| Sets

Sets are used to store multiple items in a single variable. Set items are unchangeable, but you can remove items and add new items.

When Do We Use Sets in AI?

Sets are commonly used for:

removing duplicate words building vocabularies storing labels/classes comparing features fast membership checking preprocessing datasets before training

Creating a Set

```
In [ ]: labels = {"spam", "ham", "important"}
print(labels)
```

```
{'important', 'spam', 'ham'}
```

If we add duplicate values, the set keeps only one copy:

```
In [ ]: words = {"AI", "Python", "AI", "ML"}
print(words)
```

```
{'Python', 'ML', 'AI'}
```

Suppose we want to extract only the unique words from a sentence:

```
In [15... sentence = "AI is changing AI and machine learning"

words = sentence.split()
print(words)
unique_words = set(words)

print(unique_words)

['AI', 'is', 'changing', 'AI', 'and', 'machine', 'learning']
{'machine', 'AI', 'changing', 'and', 'is', 'learning'}
```

Accessing Set Elements

Sets do not support indexing.

```
In [ ]: languages = {"Python", "Java", "C++"}

print(languages[0])
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In [5], line 3
      1 languages = {"Python", "Java", "C++"}
----> 3 print(languages[0])

TypeError: 'set' object is not subscriptable
```

```
In [ ]: models = {"CNN", "RNN", "Transformer"}

for model in models:
    print(model)
```

```
CNN
Transformer
RNN
```

Adding Elements

```
In [17... frameworks = {"TensorFlow", "PyTorch"}

frameworks.add("Scikit-learn")

print(frameworks)

{'TensorFlow', 'Scikit-learn', 'PyTorch'}
```

Removing Elements

```
In [ ]: frameworks = {"TensorFlow", "PyTorch", "Keras"}

frameworks.remove("Keras")

print(frameworks)

{'TensorFlow', 'PyTorch'}
```

Difference Between remove() and discard()

If we try to remove a value that does not exist using remove(), Python raises an error:

```
In [ ]: frameworks.remove("OpenCV")
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In [9], line 1
----> 1 frameworks.remove("OpenCV")

KeyError: 'OpenCV'
```

```
In [19... frameworks.discard("OpenCV")
```

Checking if an Element Exists

```
In [32... supported_models = {"CNN", "RNN", "Transformer"}  
print("CNN" in supported_models)  
True
```

Eng. Najwa Alghazzi